



Harness *de LLM*

o motor precisa de chassi

Modelo é o motor. Harness é o carro. Sem chassi, direção e freios, o motor não te leva a lugar nenhum.

01 O que é um Harness

A camada que transforma *chatbot* em agente

Harness é a camada de software que coloca um LLM dentro de um **loop agente** — orquestrando ferramentas, contexto, sistema de arquivos, terminal e permissões pra que o modelo execute trabalho real, não apenas gere texto.

```
prompt
├─ LLM decide
│   └─ chama tool (edit, bash, search, MCP)
│       └─ recebe resultado
│           └─ reflete e decide o próximo passo
│               └─ ... até concluir a tarefa
```

Modelos viraram commodity. A diferenciação está em como o agente pensa, edita, executa e se verifica.

02 Anatomia de um Harness

Cinco peças. *Sempre as mesmas.*

```
harness/
├─ loop agente      # plan → act → observe → reflect
├─ tool layer       # edit, bash, web, MCP, plugins
├─ context manager  # memória, arquivos, histórico, skills
├─ guard-rails      # permissões, sandbox, confirmações
└─ interface        # CLI, TUI, desktop, IDE
```

Loop > modelo

Um modelo médio com loop excelente bate um modelo top com loop ruim.

Tools são alavanca

Cada tool nova multiplica o que o agente consegue resolver sozinho.

Guard-rails são produto

Permissões mal calibradas = paranoia ou desastre. Não é detalhe.

Interface molda o uso

CLI atrai dev; desktop traz não-devs. Mesma engine, mundos diferentes.

03 Os 4 harnesses em destaque

O panorama *de bolso*

HARNESS	ORIGEM	INTERFACE	MODELO
Claude Code · Cowork	Anthropic	CLI + desktop	Claude (Sonnet/Opus/Haiku)
OpenClaw	Comunidade	CLI	Multi-provider (BYOK)
OpenCode	Comunidade	TUI no terminal	Multi-provider
Codex	OpenAI	CLI + Web/IDE	GPT / Codex

Quase ninguém usa um harness só. A combinação certa depende da tarefa.

O ecossistema *Anthropic* ponta-a-ponta

Claude Code é o CLI agente pra devs no terminal. Cowork é o modo desktop pra não-devs. Mesmo motor, públicos opostos. Free tier disponível.

✓ Skills + Plugins + MCP + Hooks Ecossistema rico e integrado, todos os componentes conversam.	✓ Permissões granulares Sandbox e modos de aprovação bem desenhados de fábrica.
✓ Refatoração multi-arquivo Agentes longos, contexto grande, edição cirúrgica entre módulos.	✓ Cowork pra não-devs Traz quem nunca abriu terminal pro fluxo agente.

Use quando: quer o melhor agente Claude, ecossistema integrado, ou precisa habilitar quem nunca abriu um terminal.

Transparência *total*, sem lock-in

Open-source. Harness comunitário compatível com Claude e outros provedores, foco em **transparência, customização e bring-your-own-key**.

✓ BYOK Sua chave, seu provedor. Sem dependência de um único fornecedor.	✓ Loop e prompts editáveis Hackeie o agente: mude o system prompt, o loop, as tools.
✓ Auditoria total Código aberto pra revisar antes de usar com dados sensíveis.	✓ Pesquisa e experimentação Ideal pra rodar local com modelos próprios ou testar setups.

Use quando: precisa de controle total, quer hackear o agente, ou rodar local. *Projeto emergente — confira o repo oficial antes de adotar em produção.*

TUI rica pra *quem vive* no terminal

Open-source. Agente de codificação com experiência TUI rica. **Provider-agnóstico** — você escolhe o modelo, ele cuida do resto.

✓ Multi-provider Roda com Anthropic, OpenAI, Groq, modelos locais — sem fidelidade.	✓ Tooling profundo Edit, shell, busca, git — todas integradas com bom default.
✓ Sessões inspecionáveis Histórico claro, compartilhável, debugável.	✓ Latência mínima Sem peso de IDE, sem GUI — só o agente e o terminal.

Use quando: o terminal é seu lar, quer uma TUI bonita e liberdade pra misturar provedores.

O CLI agente *nativo* OpenAI

Proprietário. CLI agente da OpenAI, integrado ao ecossistema GPT, com modos de aprovação granulares + companion Web/IDE.

✓ GPT/Codex recentes Acesso direto aos modelos mais novos da OpenAI sem intermediário.	✓ Approval policy Sandbox e modos de execução bem desenhados — claros e auditáveis.
✓ Infra OpenAI nativa Integração natural com o resto do ecossistema da empresa.	✓ Companions Web e IDE CLI principal, mas com superfície web e IDE pra apoio.

Use quando: já está no ecossistema OpenAI e quer um CLI agente maduro com modos de execução claros.

Você escolhe *quanto* ele decide sozinho

Read-only	Só lê, propõe diffs. Não toca em nada. Pra explorar uma base de código alheia ou auditar.
Suggest	Propõe mudanças e pede confirmação a cada passo. Modo padrão pra começar — você aprova tudo.
Auto-edit	Edita arquivos sozinho, pergunta pra rodar comandos. Bom equilíbrio entre velocidade e controle.
Full-auto	Executa tudo em sandbox. Ideal pra tarefas longas, paralelas, em ambiente isolado.

Bons harnesses deixam você escolher o nível por tarefa. Ruins ou são paranoicos demais, ou perigosos demais.

O que cada um *realmente* oferece

DIMENSÃO	CLAUDE CODE/COWORK	OPENCLAW	OPENCODE
Origem	Anthropic	Comunidade	Comunidade
Interface	CLI + desktop	CLI	TUI rica

Extensibilidade	Plugins, Skills, MCP, hooks	Hackável (código)	Plugins, providers
Multi-modelo	Claude only	Sim (BYOK)	Sim
Público	Devs + não-devs	Devs / pesquisa	Devs no terminal
Licença	Proprietário	Open-source	Open-source

Codex (OpenAI · CLI + Web/IDE · GPT only · Proprietário) não cabe na tabela acima por largura.

10 Quando escolher cada um

O *guia rápido* de adoção

✓ Claude Code / Cowork Ecossistema integrado, Skills, MCP, não-devs no fluxo.	✓ OpenClaw Transparência total, customização profunda, modelos próprios.
✓ OpenCode Vida no terminal, TUI rica, multi-provider sem lock-in.	✓ Codex Já é OpenAI nativo e quer um CLI agente maduro.
✗ Não troque toda semana Domine um, conheça os outros. Pular de barco em barco custa caro.	✗ Não escolha pelo hype Escolha pela tarefa e perfil do time. Hype passa, fluxo fica.

11 Combos poderosos

O *ganho composto* mora aqui

Harness + MCP Claude Code + MCP Linear / GitHub / Postgres → agente que lê issues, consulta dados e abre PRs sozinho.
Harness + Skills Claude Code + skills "code-review" e "weekly-report" → tarefas repetíveis viram comando único.
Harness + Subagent Claude Code dispara subagents em paralelo → revisão multi-módulo sem poluir o contexto principal.
Dois harnesses, um fluxo Cowork (planejamento + não-devs) + Claude Code (execução técnica) → produto e engenharia no mesmo loop agente.

12 Checklist de adoção

Do *zero* ao agente em produção

1. Defina a tarefa-âncora — a que você repete 5×/semana 2. Escolha o harness pelo perfil + modelo preferido 3. Comece em modo suggest — observe o que o agente faz 4. Habilite MCP/plugins só do que você realmente usa 5. Empacote padrões em Skills — não repita prompts 6. Suba autonomia gradualmente conforme ganha confiança 7. Revise mensalmente — o que automatizou? o que ainda incomoda?
--

13 Erros que todo mundo comete

Pare de fazer *isso*

Tratar harness como detalhe	É o produto. → 80% do valor prático vem dele, não do modelo cru.
Full-auto em produção	Sem confiança, é receita pra desastre. → Comece em suggest, suba autonomia aos poucos.
Ignorar MCP / plugins / skills	Você compra o motor e deixa o resto pra trás. → É onde mora o ganho composto entre tarefas.
Comparar só pelo modelo	"Sonnet vs GPT" não responde nada. → A orquestração faz mais diferença que o modelo.
Achar que um resolve tudo	Nenhum harness cobre todos os perfis. → Combine. Cada um brilha numa frente.

